

Robotics Integration Group Project I

Experimental Report

Experiment 3: Stereo Depth Estimation Using OpenCV

Sample Data

Disparity, Depth Recovery, and Point-Cloud Generation with StereoSGBM

Name: Jiasheng Gu

Student ID: 24020036020

Platform: Python 3.8.3, OpenCV 4.12.0, NumPy 1.24.4

Dataset: OpenCV aloeL.jpg / aloeR.jpg stereo pair

Abstract

This experiment implements a complete stereo depth-estimation pipeline with the official OpenCV `stereoSGBM` stereo pair. The images are already rectified, so corresponding pixels are searched along horizontal epipolar lines. `stereoSGBM` computes dense disparity, and $Z=fB/d$ converts disparity to depth for a supplied focal length and baseline. `reprojectImageTo3D` is then used to generate a colored PLY point cloud. Because the sample pair has no physical camera calibration, the demonstration uses $f=512.8$ pixels and $B=0.10$ m. The run obtains a 70.79% valid-disparity pixel ratio, a 32.13-pixel median disparity, a robust 0.776–2.153 m depth interval under the demonstration scale, and 249,334 point-cloud vertices. These metric values must be recomputed after replacing the demonstration parameters with a real stereo calibration.

Keywords: stereo vision; depth estimation; disparity; StereoSGBM; OpenCV; point cloud

1 Objectives and Dataset

1.1 Objectives

The objectives are to understand rectified stereo geometry, implement dense correspondence with OpenCV, recover depth from disparity, and export a three-dimensional point cloud. The experiment emphasizes reproducibility and a quantitative description of the generated data rather than a claimed physical distance accuracy.

1.2 OpenCV Sample Dataset

Item	Description
Files	aloeL.jpg (left) and aloeR.jpg (right)
Original resolution	1282×1110 pixels for both images
Scene	Aloe plant, flower pot, and textured background
Geometric state	Rectified sample pair; horizontal epipolar search is applicable
Data source	OpenCV official Python stereo-matching sample
Included in submission	Data/, Results/, Code/, and this report

The pair is a standard OpenCV demonstration scene with texture and depth variation. It is not a calibrated stereo-camera dataset: no physical baseline, focal length, or ground-truth depth map is distributed with the images.

2 Mathematical Principle

2.1 Rectified Stereo Geometry

After stereo rectification, corresponding points have approximately the same vertical coordinate:

$$y_L \approx y_R$$

The correspondence problem therefore reduces to a one-dimensional search along each image row. The horizontal displacement is the disparity:

$$d(x, y) = x_L(x, y) - x_R(x, y)$$

2.2 Depth and 3-D Coordinates

For a parallel stereo model with focal length f and baseline B , the depth is:

$$Z(x, y) = \frac{fB}{d(x, y)}$$

The Cartesian coordinates are:

$$X = \frac{(x - c_x)Z}{f}, \quad Y = \frac{(y - c_y)Z}{f}$$

A larger disparity means a closer point. When d approaches zero, depth becomes highly sensitive to matching noise, so the program excludes non-positive and non-finite disparities.

2.3 StereoSGBM and Q

StereoSGBM combines a matching cost with smoothness penalties over neighboring disparities. A compact energy representation is:

$$E(D) = \sum_p C(p, D_p) + \sum_{(p,q) \in \mathcal{N}} P_1[|D_p - D_q| = 1] + P_2[|D_p - D_q| > 1]$$

The run uses blockSize=5, 128 disparity candidates, uniquenessRatio=10, speckleWindowSize=100, speckleRange=32, and SGBM_3WAY mode. OpenCV stores disparity multiplied by 16; the program divides by 16. The reprojection matrix is:

$$Q = \begin{bmatrix} 1 & 0 & 0 & -c_x \\ 0 & 1 & 0 & -c_y \\ 0 & 0 & 0 & f \\ 0 & 0 & 1/B & 0 \end{bmatrix}$$

Passing disparity and Q to `reprojectImageTo3D` produces (X, Y, Z) coordinates, which are written with RGB colors to a PLY file.

3 Implementation

3.1 Environment and Parameters

Parameter	Value
Python / OpenCV / NumPy	3.8.3 / 4.12.0 / 1.24.4
Input pair	aloeL.jpg / aloeR.jpg, 1282×1110 px
Processing size	641×555 px (scale=0.5)
Matcher	StereoSGBM, SGBM_3WAY
minDisparity / numDisparities	0 / 128
blockSize	5
Demonstration f / B	512.8 px / 0.10 m

The command-line interface accepts alternative image paths, scale, focal length, and baseline, so the same program can be reused with a real calibrated stereo rig.

3.2 Processing Flow

1. Read and validate both images.
2. Resize by the same scale and convert to grayscale.
3. Compute fixed-point disparity with StereoSGBM and divide by 16.
4. Build the valid mask and calculate $Z=fB/d$.
5. Construct Q and reproject disparity to 3-D points.
6. Save visualizations, raw arrays, PLY, and JSON.

3.3 Program Code

```
#!/usr/bin/env python3
```

```
"""Stereo depth estimation with OpenCV's aloes stereo sample pair.
```

The sample pair is already rectified, so corresponding pixels are searched along horizontal epipolar lines. StereoSGBM computes disparity and the standard relation $Z = f B / d$ converts disparity to depth under a supplied baseline B and focal length f .

The OpenCV aloes pair does not ship with a physical camera calibration. The default baseline is therefore an explicit demonstration value (0.10 m) and the resulting depth scale is only meaningful after replacing it with the actual stereo baseline and focal length.

```
"""
```

```
from __future__ import annotations
```

```
import argparse
import json
import platform
from pathlib import Path
```

```
import cv2
import numpy as np
```

```
ROOT = Path(__file__).resolve().parent
DEFAULT_LEFT = ROOT / "data" / "aloeL.jpg"
DEFAULT_RIGHT = ROOT / "data" / "aloeR.jpg"
```

```
def parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(
        description="Estimate disparity, depth, and a point cloud from a rectified stereo pair."
    )
    parser.add_argument("--left", type=Path, default=DEFAULT_LEFT, help="left image path")
    parser.add_argument("--right", type=Path, default=DEFAULT_RIGHT, help="right image path")
    parser.add_argument("--output-dir", type=Path, default=ROOT / "output", help="output directory")
    parser.add_argument("--scale", type=float, default=0.5, help="image scale, e.g. 0.5 for faster processing")
    parser.add_argument("--min-disparity", type=int, default=0)
    parser.add_argument("--num-disparities", type=int, default=128, help="multiple of 16")
    parser.add_argument("--block-size", type=int, default=5, help="odd SGBM matching window size")
    parser.add_argument("--baseline", type=float, default=0.10, help="stereo baseline in metres (demo default)")
    parser.add_argument(
        "--focal-px",
        type=float,
        default=None,
        help="focal length in pixels; default is 0.8 times the resized image width",
    )
    return parser.parse_args()
```

```
def read_pair(left_path: Path, right_path: Path, scale: float) -> tuple[np.ndarray, np.ndarray]:
    left = cv2.imread(str(left_path), cv2.IMREAD_COLOR)
    right = cv2.imread(str(right_path), cv2.IMREAD_COLOR)
    if left is None:
        raise FileNotFoundError(f"cannot read left image: {left_path}")
    if right is None:
        raise FileNotFoundError(f"cannot read right image: {right_path}")
    if left.shape[:2] != right.shape[:2]:
        raise ValueError(f"left/right image sizes differ: {left.shape[:2]} vs {right.shape[:2]}")
    if not (0 < scale <= 1.0):
        raise ValueError("--scale must be in the interval (0, 1]")
    if scale != 1.0:
        size = (round(left.shape[1] * scale), round(left.shape[0] * scale))
        left = cv2.resize(left, size, interpolation=cv2.INTER_AREA)
        right = cv2.resize(right, size, interpolation=cv2.INTER_AREA)
    return left, right
```

```
def build_matcher(args: argparse.Namespace, channels: int) -> cv2.StereoSGBM:
    if args.num_disparities <= 0 or args.num_disparities % 16:
        raise ValueError("--num-disparities must be a positive multiple of 16")
    if args.block_size < 3 or args.block_size % 2 == 0:
        raise ValueError("--block-size must be an odd integer >= 3")
    window = args.block_size
    return cv2.StereoSGBM_create(
        minDisparity=args.min_disparity,
        numDisparities=args.num_disparities,
        blockSize=window,
        P1=8 * channels * window * window,
        P2=32 * channels * window * window,
        disp12MaxDiff=1,
        uniquenessRatio=10,
```

The complete source is included as `stereo_depth_estimation.py`. It also performs parameter checks, invalid-pixel handling, visualization normalization, PLY export, and JSON logging.

4 Experimental Results

4.1 Input Images



Figure 1. Left image aloel.jpg after 0.5× resizing



Figure 2. Right image aloer.jpg after 0.5× resizing

4.2 Disparity and Depth Maps

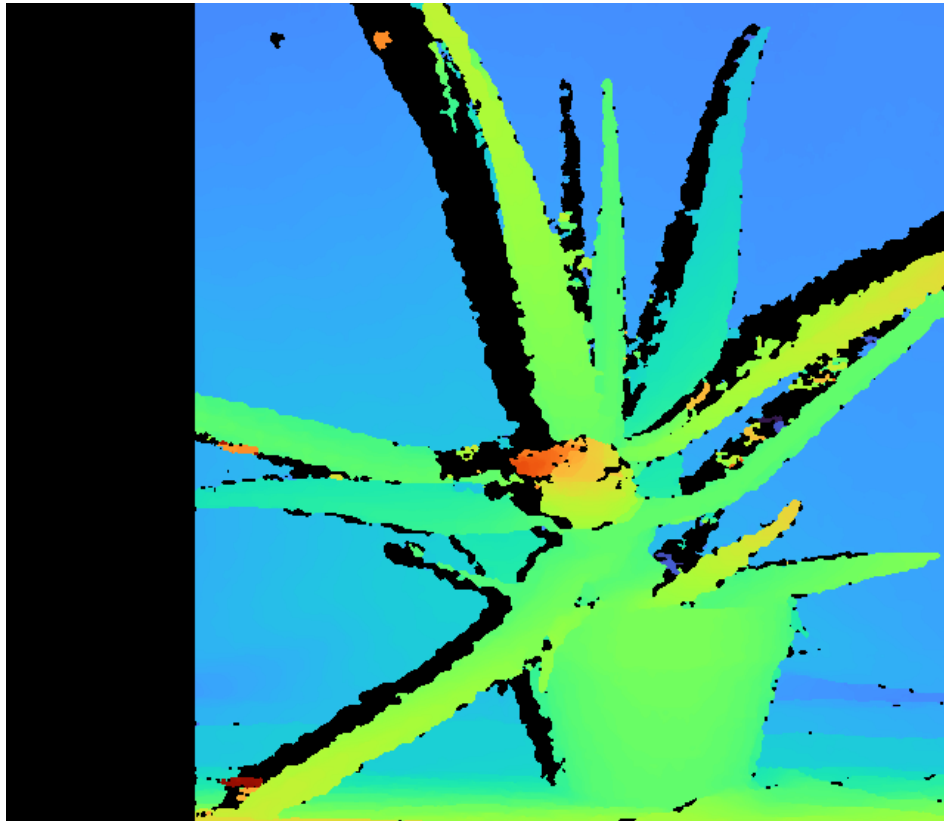


Figure 3. StereoSGBM color disparity map; black denotes invalid pixels

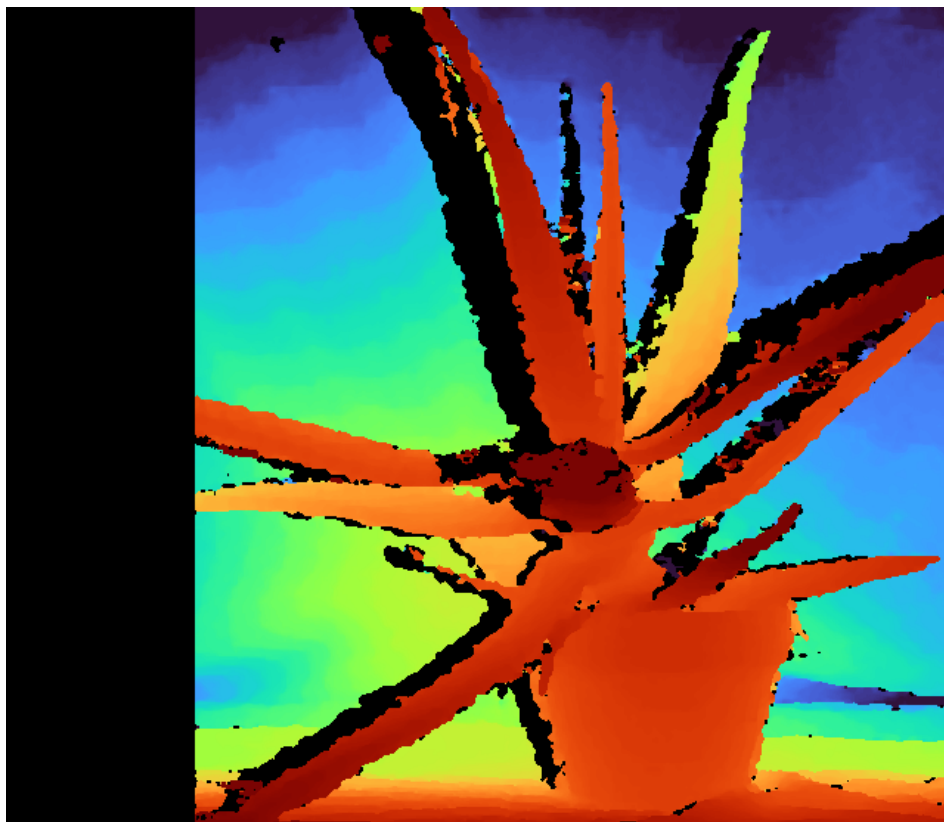


Figure 4. Color depth visualization using the demonstration f and B

4.3 Numerical Results

Metric	Result	Interpretation
Valid disparity ratio	70.79%	Finite pixels with $d > 0$
Disparity minimum	0.6875 px	Valid-region minimum
Disparity maximum	127.00 px	Search range limit
Disparity median	32.13 px	Valid-region median
Depth 5th–95th percentile	0.776–2.153 m	Robust range for $B=0.10$ m
Point-cloud vertices	249,334	Finite points written to PLY

The valid-pixel ratio is computed as:

$$\rho = \frac{N_{\text{valid}}}{N_{\text{total}}} \times 100\%$$

The depth image uses the 2nd–98th percentiles for display only; the unmodified arrays remain in `depth_arrays.npz`.

4.4 Point Cloud

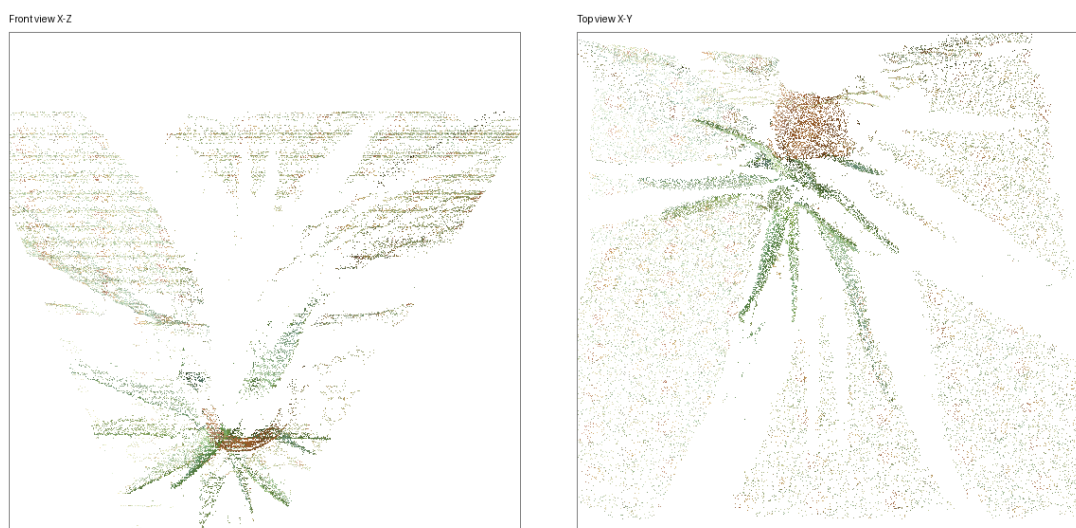


Figure 5. Front and top projections of the generated PLY cloud (random sample for display)

5 Discussion

5.1 Interpretation

The disparity map contains continuous structures on the leaves, flower pot, and background. The depth visualization separates the foreground plant and pot from the patterned background. The point-cloud projections retain the approximate plant silhouette and pot location, confirming that the disparity-to-3-D conversion is operational.

5.2 Error Sources

Source	Effect	Mitigation
Weak or repetitive texture	Holes and mismatches in background areas	Improve illumination or use a larger window
Occlusion and boundaries	No one-to-one correspondence	Use left-right consistency and edge masking
SGBM parameters	Changes smoothness and detail	Tune on a scene-specific validation set
Unknown calibration	Global depth scale may be wrong	Replace f and B with calibrated values
Small disparity	Large relative depth error	Reject near-zero d or use a wider baseline

5.3 Demonstration-Scale Limitation

The values 512.8 pixels and 0.10 m are explicit demonstration settings, not measurements of the OpenCV sample camera. For robot perception or metrology, each camera must first be calibrated, the stereo pair must be calibrated jointly, and stereoRectify/remap must be applied when the input images are not already rectified.

6 Conclusion

A reproducible OpenCV stereo-depth pipeline was implemented and tested on the official aloe sample pair. StereoSGBM produced a dense disparity image, $Z=fB/d$ produced a depth array, and `reprojectImageTo3D` generated a colored PLY point cloud. The tested run yielded a 70.79% valid-disparity ratio and 249,334 point-cloud vertices. The supplied program and report contain the formulas, source code, parameters, visual results, and output artifacts.

To use the program with a real stereo camera, replace the demonstration focal length and baseline with calibrated parameters, and perform stereo rectification if necessary. Only then can the depth values be interpreted in the physical coordinate system of the rig.

References

- [1] A. Fusiello, E. Trucco, and A. Verri, “A compact algorithm for rectification of stereo pairs,” *Machine Vision and Applications*, vol. 12, pp. 16–22, 2000. Chinese translation supplied by the user.
- [2] OpenCV, “Stereo matching sample,” `samples/python/stereo_match.py`, 4.x branch.
https://github.com/opencv/opencv/blob/4.x/samples/python/stereo_match.py
- [3] OpenCV Documentation, “StereoSGBM,” `calib3d` module.
https://docs.opencv.org/4.x/d2/d85/classcv_1_1StereoSGBM.html

Appendix A Execution Commands

```
cd stereo_depth_opencv
python3 stereo_depth_estimation.py

# use real stereo calibration parameters
python3 stereo_depth_estimation.py \
  --focal-px 700 \
  --baseline 0.060 \
  --scale 1.0
```

The output directory is `stereo_depth_opencv/output`. The submission package contains the source program, official sample images, generated outputs, and this PDF report.